Faculty of Computer Science and Applied Mathematics

Final Report

FLOCK - Emergency Evacuation Management Database

Subject: Data Structures

Prepared by: Lusine STEPANYAN

Elen YEGHIAZARYAN

Melanya MARTIROSYAN

Supervised by: Ghevond GEVORGYAN

April 27, 2026

Contents

1. Repository Link 2
2. Group Members & Contributions 3
3. Goals of the Project 3
4. Data Structures & Algorithms Used 4
   4.1 Weighted Undirected Graph (Adjacency List) 5
   4.2 Min-Heap (Priority Queue) 5
   4.3 Hash Maps / Dictionaries (Distance & Parent) 6
   4.4 Set Data Structure (Blocked Nodes & Exits) 6
   4.5 List / Array (Path Reconstruction) 6
5. Why Dijkstra Instead of BFS? (Algorithm Justification) 7
6. Complexity Analysis 8
   6.1 Graph Construction (once per building) 9
   6.2 Dijkstra's Algorithm (per route request) 9
   6.3 Blocked Node Handling 9
   6.4 Visualisation / UI Refresh 9
   6.5 Complexity Summary 9
7. Data Structures Overview 10
8. Database Overview 11
9. Overall Idea 12

1. Repository Link

GitHub Repository: https://github.com/lloocie/FLOCK.git
(Public – accessible for review)

2. Group Members & Contributions

Member	Role / Contribution
Lusine Stepanyan	Implementation of algorithms.py: adjacency set graph, edge weight dictionary, min-heap priority queue, distance and parent dictionaries, visited set, and path reconstruction list. Database design, physical implementation (SQL Server), pathfinding algorithm integration, complexity analysis, report writing.
Elen Yeghiazaryan	Implementation of building.py / building2.py: Room namedtuple, NODES dictionary, EDGES_RAW list, Euclidean distance function, EDGES_WEIGHTED list, and EXITS set. Conceptual & logical database design, normalisation, UML diagrams, relational algebra queries.
Melanya Martirosyan	Implementation of main.py UI structures: pin variable, incidents set, current_path list, current_distance variable, path_edges set, route steps list, and canvas drawing loops. Physical implementation assistance, Tkinter UI design, MVP layout, testing, documentation.

3. Goals of the Project

The goal of FLOCK is to replace static emergency signs with a real-time, intelligent evacuation system that guides each person individually to the safest and nearest exit during an emergency.

The system was designed to:

- Detect where every occupant is located inside a building at any moment
- Automatically calculate the shortest safe route to an exit — avoiding fire, smoke, or any blocked path
- Instantly adapt when conditions change (a new exit gets blocked, a new hazard appears)
- Store every incident, route, and location event as a permanent, tamper-proof audit log
- Scale across any building type — offices, hospitals, malls, airports

4. Data Structures & Algorithms Used

This section concentrates on the **core data structures** that make FLOCK work. Every structure was chosen for a specific reason related to performance, clarity, or real-time evacuation requirements.

On top of this graph, we implemented Dijkstra's Algorithm - one of the most efficient shortest-path algorithms in computer science - to find the optimal evacuation path from any starting point to the nearest available exit in real time.

Supporting data structures include:

- A min-heap (priority queue) to efficiently process the closest unvisited node at each step
- Adjacency sets for fast neighbor lookup
- Distance and parent dictionaries to reconstruct the full path once the exit is reached
- A blocked set to dynamically exclude hazardous nodes mid-evacuation

4.1 Weighted Undirected Graph (Adjacency List)

What it is: The entire building layout is represented as a graph where each room, corridor, staircase, and exit is a **node**, and every walkable connection between them is an **edge**.

Why this structure:

- Graphs naturally model physical spaces with connections
- Undirected edges allow movement in both directions (realistic for hallways)
- Weighted edges store real Euclidean distances in meters

Why adjacency list instead of matrix:

- Buildings are sparse graphs (each room connects to only 2–4 neighbours)
- Adjacency list uses $O(V + E)$ memory vs matrix $O(V^2)$
- Faster neighbour iteration during Dijkstra

4.2 Min-Heap (Priority Queue)

What it is: A binary heap that always returns the node with the smallest current distance.

Why this structure:

- Dijkstra's algorithm requires repeatedly extracting the minimum-distance unvisited node
- Min-heap gives O(log V) extraction and O(log V) insertion
- Much faster than linear search (O(V)) or sorting repeatedly

4.3 Hash Maps / Dictionaries (Distance & Parent)

What they are: Two dictionaries — one stores the best-known distance to each node, the other stores which node we came from to reconstruct the path.

Why this structure:

- O(1) average lookup and update — critical for performance
- Distance dictionary: {node: shortest_distance_so_far}
- Parent dictionary: {node: previous_node_on_path}

4.4 Set Data Structure (Blocked Nodes & Exits)

What it is: A hash set storing nodes that are currently inaccessible (fire, blocked corridor) and nodes that are valid exits.

Why this structure:

- O(1) membership test — when Dijkstra explores a node, we instantly know if it is blocked or an exit
- Dynamic updates: when a user blocks a room, we simply add it to the set — no graph rebuilding

4.5 List / Array (Path Reconstruction)

What it is: A simple Python list that stores the final sequence of nodes from start to exit.

Why this structure:

- Ordered sequence is naturally stored in a list
- O(1) append operations while building path backwards
- Easy to iterate for display and step-by-step instructions

5. Why Dijkstra Instead of BFS? (Algorithm Justification)

Criterion	BFS	Dijkstra
Edge weights	All equal (unweighted)	Supports arbitrary positive weights
Real distances	Ignores metres → may choose more steps but longer physical path	Always finds shortest physical distance
Evacuation correctness	Not guaranteed for real floor plans	Guaranteed optimal in metres
Complexity	O(V + E)	O((V + E) log V) with heap

Conclusion: In real buildings, physical distance matters more than hop count. Dijkstra guarantees the shortest metre-based route to an exit. BFS would only work if all corridors had identical length - which is never true. Therefore, **Dijkstra is the correct choice for evacuation.**

6. Complexity Analysis

6.1 Graph Construction (once per building)

- Nodes V ≈ 30–50, Edges E ≈ 40–60
- Time: O(V + E)
- Memory: adjacency list O(V + E) + weight map O(E)

6.2 Dijkstra's Algorithm (per route request)

- Implementation: heapq + set + dict
- Time complexity: **O((V + E) log V)**
    - Each node extracted from heap: O(V log V)
    - Each edge relaxed: O(E log V)
- Typical floor (V=45, E=45): ~500 operations → milliseconds
- Worst case (V=1000, E=2000): ~30,000 ops → < 1 second

6.3 Blocked Node Handling

- Blocked nodes stored in a Python set → **O(1)** membership test
- Dijkstra skips node in blocked → no graph recomputation

6.4 Visualisation / UI Refresh

- Drawing edges: O(E)
- Drawing nodes: O(V)
- Highlight path: O(path_length)
- Total per frame: **O(V + E)** → fine for 30–60 fps

6.5 Complexity Summary

Component	Complexity
Graph construction	O(V + E)
Dijkstra shortest path	O((V + E) log V)
Blocked check	O(1)
Full UI redraw	O(V + E)
Memory	O(V + E)

7. Data Structures Overview

The Python pathfinding module uses the following core data structures to model the building and compute the shortest evacuation route in real time:

- **Weighted Undirected Graph (Adjacency List)** - Each room, corridor, staircase, and exit is a node. Every walkable connection is an edge with a real Euclidean distance (in meters) as its weight. Adjacency sets provide O(1) neighbor lookup.
- **Min-Heap (Priority Queue)** - Implements the core of Dijkstra's algorithm. Always extract the closest unvisited node in O(log V) time, enabling real-time route calculation.
- **Hash Maps / Dictionaries** - Two dictionaries track state during pathfinding:
    - *Distance dictionary* - stores the shortest known distance to each node
    - *Parent dictionary* - records which node we came from to reconstruct the final path
- **Set Data Structures** - Used for two critical purposes:
    - *Blocked set* - stores nodes that are inaccessible (fire, blocked corridor). O(1) membership test allows Dijkstra to skip hazards instantly.
    - *Exits set* - stores all exit node IDs for fast termination when an exit is reached.
- **List (Path Reconstruction)** - Stores the final ordered sequence of nodes from start to nearest exit. Used to generate step-by-step evacuation instructions for the user.

Key features: O((V + E) log V) Dijkstra complexity, O(1) blocked node checking, dynamic obstacle handling without graph rebuild, and full path reconstruction for user guidance.

8. Database Overview

The SQL Server database supports the data structures by storing:

- **Buildings, floors, zones, exits, staircases** - physical layout
- **Beacons** - BLE devices for location detection
- **Users, employees, location logs** - tracking and roles
- **Incidents, evacuation routes, route steps** - emergency management
- **Nodes and edges** - the graph used by Dijkstra is mirrored in the database for persistence

Key features: 3NF normalisation, 9 triggers (block deletions, auto-set timestamps), 12 stored procedures, 7 views, 17 indexes, and 3-tier DCL access control (admin / occupant / readonly).

9. Overall Idea

Imagine you're on the 3rd floor of a hospital. A fire breaks out. You don't know the building. The nearest exit sign points to a staircase that's already filled with smoke.

FLOCK solves this.

Your smartphone detects the nearest BLE beacon - a small wireless transmitter installed on the ceiling. That beacon tells the system exactly which floor and zone you're in. The system checks the current incident, identifies which exits are still available, and runs Dijkstra's algorithm on the building's graph to compute your personal shortest safe route - step by step, room by room, in real time.

Behind the scenes, a fully normalized SQL Server database with 15 tables, 9 triggers, 12 stored procedures, 7 views, and 17 indexes handles everything: building layouts, beacon positions, user locations, live incidents, generated routes, and immutable audit logs - all secured with a 3-tier role-based access control system.

The result is a system that is smart, adaptive, fast - and most importantly, could save lives.